

CSE108 – Computer Programming Laboratory Lab #5

Date: March 27, 2026

Part 1.[20 pts] Main Function Implementation & Global Declarations and Definitions: Write the global declarations and definitions in your text editor as it is. Follow main function logic given below while implementing main function.

```
#include <stdio.h>
#include <math.h>

#define SIZE 10

/* Menu enumeration for main operations */
enum MenuOption {SIMPSON = 1, CONVOLUTION};

/* Function selection enumeration */
enum FuncChoice {POLYNOMIAL = 1, COSINE};

/* Kernel type enumeration */
enum KernelType {EDGE_DETECTION = 1, SMOOTHING, SHARPENING};

/* Error enumeration for integration calculation */
enum ErrorType {SUCCESS, INVALID_BOUNDS, NOT_POSITIVE, NOT_EVEN};

/* Mathematical functions that will be passed as parameters to simpson function */
double polynomial(double x);
double cosineFunc(double x);

/* Function prototype for Part 1 Solution */
double simpson(double f(double), double a, double b, int n, enum ErrorType *error);

/* Function prototype for Part 2 Solution */
void convolution(int arr[], int size, int kernel[3], int result[]);
```

```
int main()
{
    /* Define enum variables for each enum created above */

    /* Define an integer array with size 10. Its values are 1,3,2,5,4,7,6,9,8,10 */
    /* Define an integer array with size 10. Do not assign any value to it yet. */

    /* Define an integer array with size 3. Its values are 1,0,-1. It is edge detection kernel. */
    /* Define an integer array with size 3. Its values are 1,2,1. It is smoothing kernel. */
    /* Define an integer array with size 3. Its values are -1,2,-1. It is sharpening kernel. */

    /* Define double variables for lower bound, upper bound and final integral result */
    /* Define an integer variable for number of intervals */

    /* Print Main Menu and ask user its choice */

    /* Use switch. One case is SIMPSON, other case is CONVOLUTION. For default case, print "Invalid" */

    /* Inside SIMPSON case, ask user which function to integrate. */
    /* Inside SIMPSON case, ask user lower bound, upper bound, number of intervals */
    /* Inside SIMPSON case, use switch. Its cases must be POLYNOMIAL and COSINE */
        /* Inside POLYNOMIAL case, call simpson function and pass polynomial function as argument */
        /* Inside COSINE case, call simpson function and pass cosineFunc function as argument */
    /* Inside SIMPSON case, check the value of enum ErrorType variable. If it is SUCCESS, print integral result */
    /* Inside SIMPSON case, check the value of enum ErrorType variable. Print its value like "Status: NOT_EVEN" */

    /* Inside CONVOLUTION case, Print kernel selection menu. Ask user which kernel they choose. */
    /* Inside CONVOLUTION case, use switch. Its cases must be EDGE_DETECTION, SMOOTHING, SHARPENING */
        /* Inside EDGE_DETECTION case, call convolution function with appropriate kernel array with values 1,0,-1. */
        /* Inside SMOOTHING case, call convolution function with appropriate kernel array with values 1,2,1. */
        /* Inside SHARPENING case, call convolution function with appropriate kernel array with values -1,2,-1. */
    /* Inside CONVOLUTION case, print result array that was sent to the convolution function as argument and filled there.
}
}
```

Part 2.[40 pts] Integral Calculation with Simpson’s Rule: Implement Simpson's Rule formula inside **simpson** function to calculate integral. **a, b and n values are get from the user.** Follow the code below.

$\int_a^b f(x)dx \approx \frac{h}{3} \left[f(a) + f(b) + \sum_{i=1}^{n-1} c_i \cdot f(x_i) \right]$ $x_i = a + i \cdot h$	$c_i = \begin{cases} 4 & \text{if } i \text{ is odd} \\ 2 & \text{if } i \text{ is even} \end{cases}$ $h = \frac{(b - a)}{n}$
--	---

For example, if **a=2, b=5 and n=8**, the integral result will be equal to: **(h/3) * [f(a) + 4*f(x1) + 2*f(x2) + 4*f(x3) + 2*f(x4) + 4*f(x5) + 2*f(x6) + 4*f(x7) + f(b)]** And x1 = a + 1*h , x2 = a + 2*h, x3 = a + 3*h, x4 = a + 4*h, x5 = a + 5*h, x6 = a + 6*h, x7 = a + 7*h.

```
/* ===== Mathematical Functions (can be passed as parameters) ===== */

double polynomial(double x) { return x * x * x + 2 * x + 1; }

double cosineFunc(double x) { return cos(x); }

/* ===== Simpson's Rule Implementation ===== */
double simpson(double f(double), double a, double b, int n, enum ErrorType *error)
{
    /* Check bounds a and b. If a is not smaller than b, set error INVALID_BOUNDS. */
    /* Check n. It must be positive. Otherwise, set error NOT_POSITIVE. */
    /* Check n. It must be even for Simpson's Rule. Otherwise, set error NOT_EVEN. */
    /* Define h, sum, x and calculate h. */

    /* Calculate x and sum value inside one for loop. */

    /* Set the error variable to SUCCESS */
    /* Return (h / 3.0) * sum */
}
```

Part 3.[40 pts] Convolution Filter for 1D Array: Think of the kernel as a window sliding over the input data. At each step, align the kernel with a segment of the array. Move the kernel from left to right one position at a time. Multiply the corresponding elements and accumulate the sum to generate a single output value.

<pre>/* ===== Convolution Filter ===== */ void convolution(int arr[], int size, int kernel[3], int result[]) { /* Loop through result array from i = 0 to its last index */ /* Inside loop, fill result[i] using arr array's values */ }</pre>		
<pre>===== MAIN MENU ===== 1. Integral Calculation with Simpson's Rule 2. Convolution Filter for 1D Array Enter your choice: 1 Select function to integrate: 1. f(x) = x^3 + 2x + 1 2. f(x) = cos(x) Enter function choice: 2 Enter lower bound a: 0 Enter upper bound b: 0.785398 Enter number of intervals n: 200 Approximate integral = 0.707107 Status: SUCCESS!</pre>	<pre>===== MAIN MENU ===== 1. Integral Calculation with Simpson's Rule 2. Convolution Filter for 1D Array Enter your choice: 2 Select kernel type: 1. Edge Detection [1, 0, -1] 2. Smoothing [1, 2, 1] 3. Sharpening [-1, 2, -1] Enter kernel choice: 1 Selected Kernel: [1, 0, -1] Convolution result: -1 -2 -2 -2 -2 -2 -2 -1</pre>	<pre>===== MAIN MENU ===== 1. Integral Calculation with Simpson's Rule 2. Convolution Filter for 1D Array Enter your choice: 1 Select function to integrate: 1. f(x) = x^3 + 2x + 1 2. f(x) = cos(x) Enter function choice: 1 Enter lower bound a: 5 Enter upper bound b: 17 Enter number of intervals n: 99 Error: n must be even for Simpson's Rule! Status: NOT_EVEN</pre>